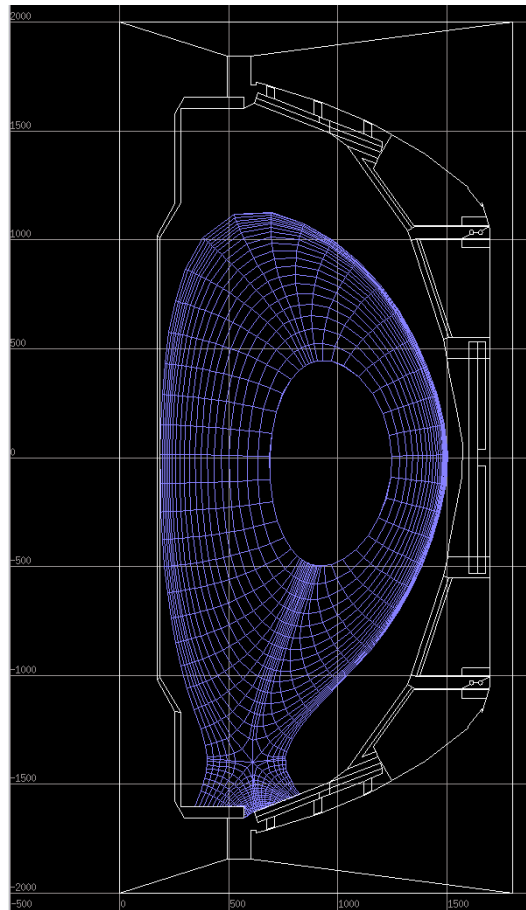


Abstract

This is the user's manual for the **DG** geometry definition code. Its primary objective is to simplify the specification of the geometry, and other input data, required for the simulation of the boundary of magnetic fusion devices. The most straightforward application of the code is to divertor tokamak geometries (having one or two X-points), although in principle other magnetic configurations are possible. The flexible design of **DG** allows its output to be used with multiple magnetic fusion simulation codes, including **B2-Eirene** and **DEGAS 2**. In fact, some of the objects defined in **DG** and described in this manual may be designed specifically for use by these codes.



User's Manual for DG Version 2.10

Konstantin Kukushkin, Andrei Kukushkin
ITER Organization
Alan Chin, Daren Stotler
Princeton Plasma Physics Laboratory

October 13, 2009

Chapter 1

Introduction

1.1 Purpose of DG

DG is the backbone of the **Interactive Graphical Interface** package. It allows user to visualize the model geometry, to modify it, and to specify the geometry-related data necessary for a grid generator and modelling codes. It was originally designed to alleviate transfer of the divertor geometry from the CAD system to the **B2-Eirene** code package. However, it has proved to become a powerful tool of more general purpose, allowing one, e. g., to specify arbitrary shape of the divertor structures, to check visually the quality of the computational grids produced by a mesh generator, to analyse the error messages from the Monte Carlo routine, to assess easily the compatibility of the plasma equilibrium used in the calculations and the wall geometry of the machine, etc. It can also be very efficient as a sort of very simple drawing program to design the problem geometry from scratch, and not only for B2-Eirene. With use of this code, the most boring and time-consuming part of the preparatory work necessary for the edge plasma modelling in a realistic geometry, becomes really a pleasure. Try it and see yourself!

1.2 DG objects

1.2.1 External objects

The external objects are the files which DG imports or exports. They normally have default extensions by which are used by the code when displaying the lists of files in the I/O control windows. The following table gives a brief description

of the files used in DG.

Input files:	Description	Extension
DG model	All the data relevant to the model	.dg
Template	Geometry data from CAD drawing or other source	.ogr
Equilibrium	Values of the poloidal magnetic flux on a regular grid	.equ
Mesh	The computational grid to be used by B2-Eirene	
Output files:		
DG model	All the data relevant to the model	.dg
Output file	Specification of the <i>elements</i> and some other information – to be used by the interface routines producing the input files for B2-Eirene.	.dgo
Targets file	Specification of the <i>targets</i> – to be used by the interface routines producing the input files for B2-Eirene and Sonnet / Carre.	.trg
Structure file	Specification of the structures – to be used by the interface routines producing the input files for Sonnet / Carre.	.str
Configuration file:		
	The file containing information on the DG <i>layers</i> and <i>variables</i> .	.dgc

1.2.2 Internal objects

The internal objects are those with which you deal when working with DG.

Objects	Description
Drawing objects	
Points	Just points (displayed as red squares) which can be used to create <i>elements</i> or for diagnostic purpose.
Irregular points	Every point which is not regular. A regular point connects exactly two <i>elements</i> having the same orientation (see <i>External normals</i>)
Elements	Straight line segments (displayed in bright white) connecting <i>points</i> . Normally, they represent material surfaces in the neutral transport code (“additional surfaces” in Eirene parlance). You can edit elements (create, delete, split, join, change orientation, etc.) and assign <i>variables</i> to them.
Surfaces	Contours of constant poloidal flux (displayed in yellow) computed from <i>equilibrium</i> data. You can create, move, and delete them. The surfaces allow you to specify the requirements on the radial mesh for the mesh generator (Sonnet or Carre), and also to check the compatibility of your wall structures with the magnetic configuration.
Grid points	Points on the separatrix which represent partition of the magnetic surfaces in different regions for the grid generator (Sonnet or Carre). They are displayed as dashes in magenta; the separatrix being drawn in red.
Separators	Straight line segments (displayed in cyan) connecting the outer nodes of the computational <i>mesh</i> with surrounding <i>structure</i> . The separators also represent additional surfaces, but you can only re-position their outer edges. Their purpose is separation of <i>additional cells</i> – in terms of Eirene. As of V. 1.4 of DG, “separators” have been temporarily disabled.

Sources	Special points which can be grouped together (displayed as white asterisks). They can be used to specify the point sources of neutrals, – and also in calculation of radiation load onto the first wall.
Equilibrium	A simple visualisation of the equilibrium data – red and blue dashes corresponding to negative and positive, respectively, values of the poloidal magnetic flux.
Mesh	Computational mesh, displayed in blue, produced with a grid generator (Sonnet or Carre) and imported into DG. Allows you to check the quality of the mesh and to make the necessary corrections to the input data for the grid generator.
Chords	Straight line segments representing the diagnostic chords for Eirene and b2plot.
External normals	Are displayed as magenta dashes and depict the orientation of the elements. They point to the right as you move from the beginning to the end of an element. This is the direction of the external normal in terms of Eirene.
Template	Imported graphics from external source (e.g. from CAD drawing or from old Eirene input file). The template is displayed in cyan, and it can be converted into <i>elements</i> .
Axes	Co-ordinate axes which can be displayed on the drawing.
Grid	Coordinate grid which can be displayed on the drawing.
X-point	The X-point of the magnetic equilibrium; it effectively separates the divertor from the core plasma. In double null configurations, there are two X-points which may or may not be on the same flux surface. The code must be told the topology of the equilibrium (see <i>topology</i>) in order to locate the X-points. This must be done before specifying <i>surfaces</i> or <i>grid points</i> .

Topology	The topology of the magnetic separatrix. A set of commonly used topologies is provided with DG. Once the <i>equilibrium</i> and topology have been imported, the code will attempt to locate the X-point(s); if successful, the separatrix (or separatrices) will be drawn in red.
----------	--

Additional data

Variables	Typically used to associate various pieces of data with the <i>elements</i> . This information can be then passed to other codes via the output files.
Layers	Groups of variables.
Targets	Bound the <i>surfaces</i> in the poloidal direction, and thus must be specified for the grid generator. They are thus analogous to the actual divertor targets in a tokamak.
Structure	Bound the <i>surfaces</i> in the radial direction. They are thus analogous to the walls of the vacuum vessel in a tokamak.
Target edges	The elements which should be reconnected to the corresponding corners of the calculational mesh in order to avoid mismatches of the mesh and the targets in the Monte Carlo calculations.

Chapter 2

Using DG

2.1 Starting DG

To start DG with a blank window, type:

dg

at the UNIX prompt.

To start DG with an existing model (or several models) loaded, type:

dg <filename.dg> [...]

After you start DG, the main window will appear.

2.2 Main window

After you start DG, the main window appears. It has four areas: menu bar, toolbar, work area and the message line.

2.2.1 Menu bar

The menu bar gives you access to the most of the DG functionality. It contains several buttons, which can be used to open the corresponding menus by clicking the left mouse button on it. Every menu can be torn off and left on the screen after opening it. In order to do this, just click at the dashed line at the top of the menu window. In this document, we will show menu items in **bold typewriter** font.

The following menus are available at present;

- File** You use this to start a new model, to work with external objects (to save or open the model; to import templates, equilibrium files, topologies, or computational meshes; to produce the output files which are used for generation of the input files for computational codes), or to quit the application.
- Edit** Allows you to perform some global operations, such as recovering from errors, selecting and de-selecting all elements, creation or removal of groups of objects, and shift or rotation of the picture.
- View** Here you can control the overall view of the model and specify which objects should be visible on the screen.
- Commands** Allows you to perform global commands on the model such as converting a *template* to *elements* and renumbering the elements.
- Variables** You will use this to specify additional geometry-related information for the output files (eg., location of targets, boundary conditions for the Monte Carlo routine, etc.).
- Options** From this menu you can tailor the main window and operating mode to fit you preferences. It also allows you to customize the DG variables (*only for experienced users!*).
- Window** Allows you to open more windows with different views of the same model, to close the current window, to open a window showing the statistics for the model, and to open the **Toolbox**.
- Help** This is the way to get some online help on DG and to identify its current version.

2.2.2 Toolbar

The toolbar allows you to assign different *tools* to each of the mouse buttons. There is a separate menu for each mouse button, each containing the list of available tools.

2.2.3 Work area

The work area is the area where the drawing is displayed. Here you can move around the drawing, and you can use *tools* to edit it. The work area can optionally display coordinate axes and coordinate grid to simplify positioning of objects.

2.2.4 Message line

The message line is located below the work area. It is used to display error messages and information on the objects. Error messages are sometimes displayed in an error box instead of the message line, and in this case you must click “OK” to proceed.

2.3 Using tools

Tools are used to perform actions in the work area. They allow you to create, move, modify, or remove objects, to query information on the objects and to move around the drawing. In this document, we will indicate tools in *italic typewriter* font.

A tool *must* be assigned to a mouse button in order to be used. You can select the tool to be assigned to each of the mouse buttons either from the toolbar, or from the **Toolbox** (accessible from the **Window menu**). There is one small difference between these two options. Operating with the toolbar, only the left mouse button should be used to select the tools associated with the three mouse buttons, On the other hand, when working with the **Toolbox**, a tool is associated with a given mouse button by clicking that button on the tool.

To use a tool, move the mouse pointer into the work area and depress the mouse button to which the tool has been assigned. Then, you can optionally drag the mouse. To complete the mouse drag, release the mouse button with the mouse pointer still inside the work area. To cancel the action associated with the drag, move the mouse pointer out of the work area and release the mouse button.

Some tools require you to ‘select’ an object. To select an object, move the mouse pointer to the object and depress the mouse button. The object selected will be highlighted in another color. The *green* color means OK, while the *red* one indicates that the operation cannot be done. Normally, some explanatory text appears in such cases on the message line.

2.4 Using Undo

You can reverse the effect of almost any operation by issuing the **Edit | Undo** menu command. You can also reverse the effect of Undo command by **Edit | Redo**. The DG keeps track of practically all the operations you did starting from opening the window, and thus you can recover from almost all the errors you could make in the course of the model editing. To reverse the effect of all the Undo commands you issued, use **Edit | Redo all**.

Chapter 3

Working with models

3.1 Opening an existing model

To open an existing DG model from DG, use the **File** | **Open...** menu command. A dialog box will appear prompting you for a filename. You can either select a file from the list or type the filename in the edit field. Then press **Ok**. A new window displaying the selected model will be opened. If you issue the **File** | **Open...** command immediately after starting DG, the file will be loaded in the current window. If an error happens, the error box appears. Press **Ok** to close the error box.

To start DG with an existing model loaded, specify the filename on the DG command line. If you specify multiple filenames, DG will open a separate window for each model.

3.2 Creating a new model

To create a new, empty model with DG, use the **File** | **New** menu command. A new blank window will be opened.

When you start DG without parameters, the DG window is blank.

3.3 Saving a model

To save a model, use the **File** | **Save...** menu command. A dialog box will appear prompting you for a filename. If the current model has a filename assigned

to it, this filename will appear in the edit field. In order to save the model under a different name, or to save the newly-created file, you should either enter the new name in the edit field, or select a name from the list (to overwrite an existing file). After that, press **OK**. An autosave feature automatically saves a version of the file with a “~” appended to the name every 5 minutes (see **Options | Save interval**); of course, a new file must be saved before this can work. If you need to use this file (e.g., in the event that DG crashes unexpectedly), you should rename this file to remove the “~”.

3.4 Quitting DG

To quit DG, you must close all windows.

To close a window, issue the **Window | Close** menu command. You can also use the window manager to close a window click the “X” at the top right of the menu. If you’ve made changes to the model in this window, a warning box will appear. Select **Save** to save the model. Select **Discard** to close the window without saving changes. Select **Cancel** to leave the window open.

To close all windows, use the **File | Exit program** menu command.

There is a caveat in the current version of DG that iconified windows containing unsaved work are not de-iconified after these commands. You will have to de-iconify them manually in order to see their warning boxes. All other iconified windows will be closed without problems.

Chapter 4

Working with elements

4.1 Converting a template

It is often more convenient to convert a template into a set of elements and then edit them to suit your needs, rather than to define all elements by hand.

First, you should import the template. Choose the **File | Template...** menu command. A dialog box will appear prompting for a filename, similar to that for opening DG files. After importing the template, choose **View | Picture view** to adjust the zoom factor in order to make the template visible. Enable **View | Axes** and **View | Grid** to display the coordinate system.

DG lets you adjust the template position by moving it by a fixed distance and/or rotating it 90 degrees around (0,0). To adjust the position, choose the **Edit | Move/Rotate...** menu command. A dialog box will appear. Enable the **Template** check box. To move the template, enter X- and Y-increments and press **Move**. To rotate, press **Rotate**. If the template moves out of sight, again choose **View | Picture view**. Press **Close** when done.

To convert the template into a set of elements, choose **Commands | Convert template**.

Note that after the conversion, the template remains visible. This is convenient for editing elements, because the difference from the original template is visible. You can hide or display the template by disabling or enabling **View | Template**. To unload the template permanently, choose **Edit | Delete | Template**.

Elements produced in this manner are sometimes improperly connected or have wrong external normals. Manual editing of the elements may be required to

remedy these problems.

4.2 Creating elements manually

There are several tools for creating new elements.

Use *Add element* to create the first element or to attach an element to a pre-defined point. Move the mouse pointer to the starting point and press the mouse button. Then move the pointer to the ending point and release the mouse button.

Use *Split element* to split an element into two. Select an element. It will split into two. Drag the newly created point to the desired location and release the mouse button.

Use *Connect points* to create an element between two predefined points. Select the first point, move the mouse cursor to the second one and release the mouse button. You cannot create more than one element between the same two points.

You can also use the **Edit | Create | Point...** menu command to create points that will be later connected using the *Connect points* tool. A dialog box will appear. Enter the desired coordinates and press **Create**. The dialog box will remain on the screen, allowing you to create more than one point without reopening it. To close the dialog box, press Cancel.

4.3 Editing elements

There are several tools for modifying existing elements.

Use *Delete* to remove an element. Select the element and release the mouse button. Deleting all of the elements connected to a given point also deletes that point.

Use *Join elements* to join two adjoined elements into one (reverse effect of *Split element*). Select the point between the elements and release the mouse button. You can only join elements around a regular point. There must be exactly two elements attached and their external normals must point to the same direction.

Use *Move* to move a point with all attached elements. Select the point and drag it to the new location. Then release the mouse button.

Use *Reposition* to attach one end of an element to another point. Select the element near one of its ends, move the mouse pointer to another pre-defined

point and release the button.

4.4 Changing external normals

Having the external normals of adjacent elements aligned permits them to be easily marked; some operations also require the normals of adjacent elements to be aligned (i.e., for the points involved in the operation to all be regular).

To display or hide external normals, enable or disable **View | External normals**.

Use the *Reverse normals* tool to change directions of external normals. Select an element to change its normal. Moving the mouse pointer to adjacent elements will force their normals to point to the same direction as that of the first element. Release the mouse button to end the action.

4.5 Renumbering elements

Each element has a integer label associated with it. These determine the order in which the elements are written into the output file. This feature is very useful for debugging. DG uses an internal counter to assign these numbers. Each time the user creates an element, the counter is incremented by one and its value is used as the element number. However, when the user deletes an element, the remaining elements are not renumbered and the counter is not decremented. Since many unused numbers consume disk space and may slow down programs that use DG output, these unused element numbers should be periodically eliminated. Renumbering elements is absolutely safe except when you are doing some debugging and need these numbers to identify elements in the output file.

To display or hide element numbers, enable or disable **View | Display | Numbers**.

To count unused numbers, choose **Window | Statistics** in order to display information about the current file. The 'unused numbers' entry contains this count.

To eliminate unused element numbers, choose **Commands | Renumber elements**.

Chapter 5

Controlling how objects are displayed

5.1 Changing the zoom factor and location

You can use the *Zoom* tool to move around the drawing and to change the zoom factor. You can also use commands from the **View** menu.

To move around with the *Zoom* tool, press and release the mouse button anywhere in the work area without moving the mouse. The work area will be adjusted so that the location where the button was pressed is moved to the center. Alternatively, you can press Shift+mouse button, move the mouse pointer and release the button. The work area will be adjusted so that the location where the button was pressed is now shown where the button has been released.

To change the zoom factor with the *Zoom* tool, press the mouse button, move the mouse in order to draw a rectangle around the area to be shown in the work area. Then, release the mouse button. Note that the area actually shown may be enlarged in order to preserve the aspect ratio.

The **View** | **Zoom in** command increases the zoom factor.

The **View** | **Zoom out** command decreases the zoom factor.

The **View** | **Picture view** command adjusts the zoom factor and location so that all objects fit into the work area. This command ignores objects that are hidden because of settings in the **View** menu.

The **View** | **Selection view** command adjusts the zoom factor and location so that all marked objects fit into the work area.

5.2 Hiding and displaying objects

Toggle buttons in the **View** menu let you hide or display the coordinate system and various kinds of objects.

Note that you cannot select an object that is not displayed. In particular, you can only select points that are displayed (a small square is drawn around the point). The **View | Points** setting controls the display of points. When it is disabled, the **View | Irregular points** setting controls the display of irregular points. It is convenient to display only irregular points while correcting a converted template, because irregularities can be seen very easily.

5.3 Rotating the view

The display may now be viewed at any angle of rotation. The user may specify the desired angle by using the Rotate tool, the commands under the **View | Rotate** menu, or the equivalent keyboard shortcuts to increment (“Ctrl+”), decrement (“Ctrl+”), or zero the angle (**Rectify**, shortcut “Ctrl+C”); see Fig. 5.1. The rotation does not affect the underlying structures being displayed, but only the drawing procedure. The **Previous view** command will undo rotations. Rotations are saved starting with file format version 115. Files of older formats are still compatible; the default angle will be 0. Note that the Picture View command (**View | Picture View**) zeroes the rotation angle.

5.4 Stretching and shrinking the view

The aspect ratio may be “unlocked” by enabling the **View | Mode | Stretch** button. In Stretch view mode, the **View | Stretch/Shrink** menu item becomes active (see Fig. 5.2), and the user may use these tools to stretch or shrink the x and y axes. To stretch or shrink the current view by a factor of 2, use the first four buttons of this menu. The keyboard shortcuts are “Ctrl+Shift+H” to stretch horizontally (in the x direction), “Ctrl+H” to shrink horizontally, “Ctrl+Shift+V” to stretch vertically (in the y direction), and “Ctrl+V” to shrink vertically. Precise stretching or shrinking factors can be specified with the **Stretch...** dialog. The aspect ratio can be reset with the **Justify** menu item. Additionally, zooming with the highlighted rectangle now zooms to display exactly the contents inside the rectangle, rather than expanding to ensure the proper aspect ratio. Dis-

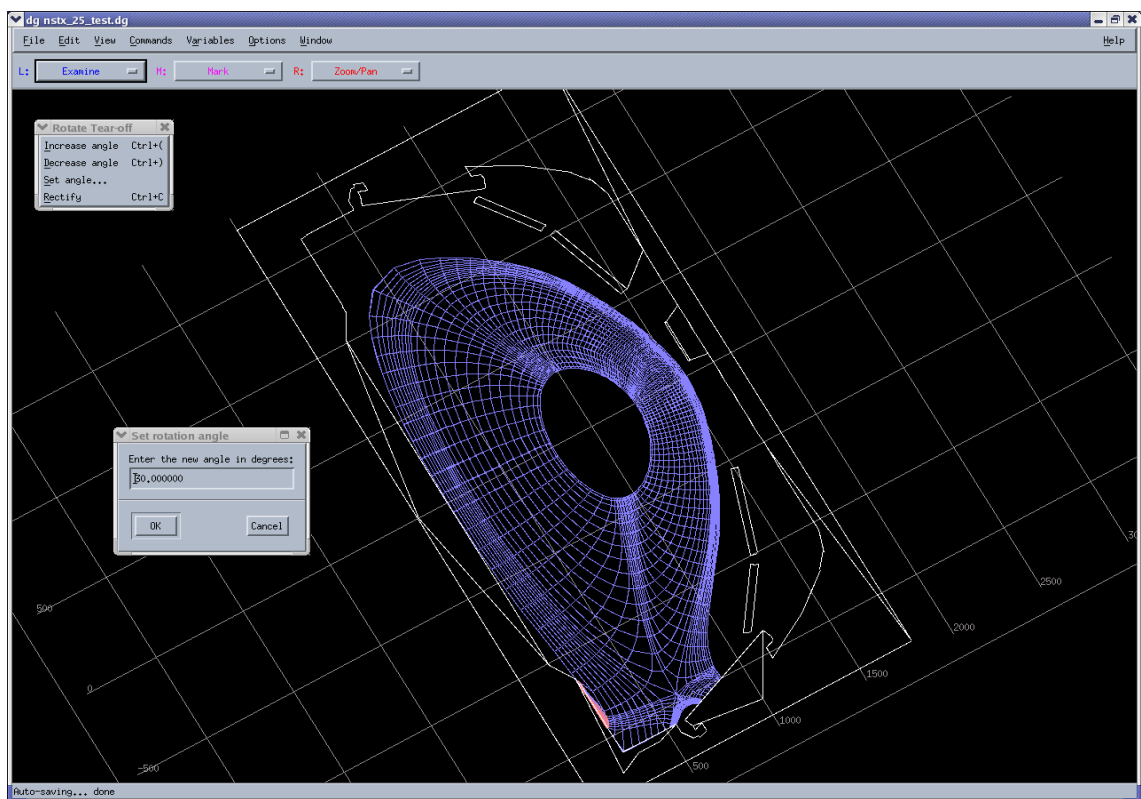


Figure 5.1: Demonstration of a rotated view with associated dialog box and tear-off menu.

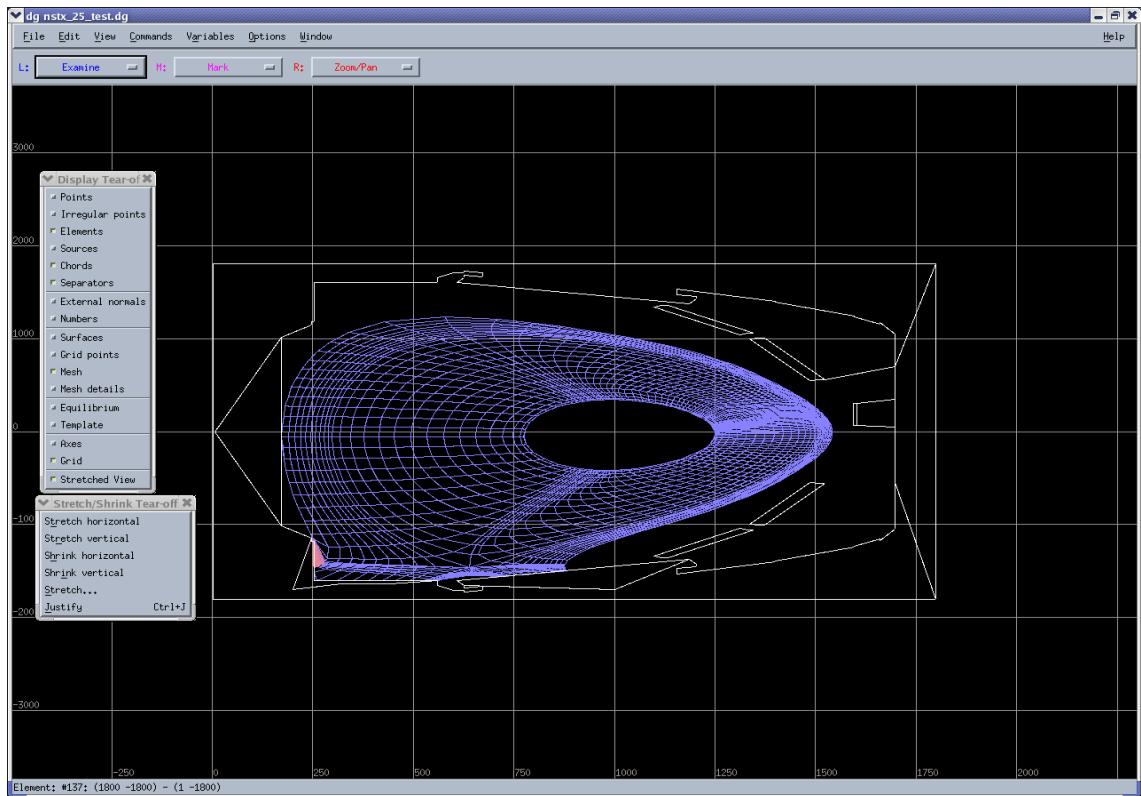


Figure 5.2: Demonstration of the variable aspect ratio capability along with the associated tear-off menu.

abling Stretch mode redraws the display with proper aspect ratio and preserves the ratio in future changes of view.

The *Stretch* tool works a little differently since it can stretch either horizontally or vertically. If the mouse is within 45° of an imaginary horizontal line through the center of the window, a mouse drag will stretch or shrink the view in the horizontal direction. Likewise, if the mouse is within 45° of the vertical line through the center of the window, a mouse drag stretches or shrinks in the vertical direction. If **Stretch mode** is disabled, the *Stretch* tool does nothing.

Note that the Picture View command (**View** | **Picture View**) resets the aspect ratio, but does not disable Stretch mode.

5.5 Top-down view

To facilitate visualization and specification of Chords with nonzero toroidal extent (see Ch. 7), a view from above the torus is now available via the **View | Mode | Top-down view** button. When this button is enabled, the main window depicts the view of the present geometry looking down along the y axis. The elements in the model are assumed to be toroidally symmetric. In this view, the elements are displayed as 48 (arbitrarily chosen) straight line segments. The only other objects currently displayed in this view are chords, although working drawing procedures exist for all markable objects. Since displaying every element in the model would likely result in confusion, the user must select the specific elements to be depicted. The variable **Top View Elements** has been defined for this purpose (**Variables | Top View Elements**). The appearance of chords (with some toroidal extent) can be toggled with either **View | Display | Chords** or **View | Display | 3dchords**. As in the normal (poloidal) view, **View | Display | Elements** toggles the appearance of elements in the top view. Since top view was intended to be used only for chords, many tools are disabled in this mode. Likewise, since top view uses the same screen coordinates as the normal view, it may be necessary to rezoom after switching between the two views. **View | Picture View** can be used for this purpose in either mode.

Chapter 6

Working with magnetic surfaces and grid points

6.1 Equilibrium conversion and manipulation

The DG/equtrn directory has some utilities for converting between various widely used formats for magnetic equilibrium files and for making minor modifications to them.

6.1.1 d2d

This is a script that invokes the **dg2dg** executable. Its sole function is to double the resolution of a DG equilibrium file. To invoke it:

```
d2d <filein>.equ <fileout>.equ
```

6.1.2 d2e

This is a script that invokes the **dg2ef** executable to convert a DG equilibrium file to the EFIT format (specifically, the EFIT “g” file). This is invoked with (the file suffixes are recommended for clarity and are not enforced):

```
d2e <filein>.equ <fileout>.efit
```

Some older DG files lack the magnetic field data needed for this conversion. The default name for the file containing its value is **field.snn** (i.e., using the above

invocation). Alternatively, the name of the file containing the field can be specified as a third argument:

```
d2e <filein>.equ <fileout>.efit <fieldfile>
```

6.1.3 d2v

This is a script that invokes the **dg2vr** executable to convert a DG equilibrium file to the Carre (or TdeV) equilibrium format. This is invoked with (the file suffixes are recommended for clarity and are not enforced):

```
d2v <filein>.equ <fileout>.v
```

Some older DG files lack the magnetic field data needed for this conversion. The default name for the file containing its value is **field.snn** (i.e., using the above invocation). Alternatively, the name of the file containing the field can be specified as a third argument:

```
d2v <filein>.equ <fileout>.v <fieldfile>
```

6.1.4 e2d

This is a script that invokes the **ef2dg** executable to convert an EFIT format (specifically, the EFIT “g” file) equilibrium to the format read by DG. This is invoked with (the file suffixes are recommended for clarity and are not enforced):

```
e2d <filein>.efit <fileout>.equ
```

6.1.5 n2d

This is a script that invokes the **nk2dg** executable to convert an equilibrium in the “Naka” format to the format read by DG. This is invoked with (the file suffixes are suggested for clarity and are not enforced):

```
e2d <filein>.nk <fileout>.equ
```

6.1.6 rps

This is a script that invokes the **risepsi** executable. Its sole function is to increase the poloidal flux values in the DG equilibrium file by the specified amount. To invoke it:

```
rps <filein>.equ <fileout>.equ <increment>
```

6.1.7 v2d

This is a script that invokes the **vr2dg** executable to convert an equilibrium in the Carre (or TdeV) format to the format read by DG. This is invoked with (the file suffixes are recommended for clarity and are not enforced):

```
v2d <filein>.v <fileout>.equ
```

By default, this script will look for the value of the magnetic field in a file **field.snn**. Alternatively, the name of the file containing the field can be specified as a third argument:

```
v2d <filein>.v <fileout>.equ <fieldfile>
```

6.1.8 cropequ

This is an executable (no corresponding script) that crops a DG equilibrium file. The code is invoked with: (the file suffixes are recommended for clarity and are not enforced):

```
cropequ <filein>.equ <fileout>.equ
```

The code will print out the dimension and spatial extent of the input equilibrium grid then request the values for the output grid in the order: number of radial grid points, minimum radius, maximum radius, number of vertical grid points, minimum vertical coordinate, maximum vertical coordinate. The maximum number of grid points is 257 (can be increased via a parameter statement in the code). The code will require the requested spatial extent to lie within that of the input grid. If it does not, the offending parameter is reset to match the corresponding value from the input grid.

6.1.9 prinequ

This is an executable (no corresponding script) that prints out the contents of a DG equilibrium file in tabular format. The file is read from standard input and the result is sent to standard out. So, the preferred invocation would be (the file suffixes are recommended for clarity and are not enforced):

```
prinequ < <filein>.equ > <fileout>.txt
```

6.2 Importing equilibrium

An equilibrium file must be imported before defining any surfaces or grid points.

To import an equilibrium file, choose **File | Import | Equilibrium...**. A dialog box will appear prompting for a filename. Select a file and press **Ok**. Select **View | Picture view** to make sure the equilibrium grid is visible in the work area. Red color represents positive ψ values, while blue represents the negative ones.

6.3 Output mode

The detailed procedures for creating and editing surfaces and grid points differ slightly for the Sonnet and Carre mesh generators. So, before undertaking these tasks, the user should select the appropriate output mode in **Options | Output mode**. Since only the Carre code is distributed along with this manual, only procedures pertinent to it will be described.

This dialog box also allows the user to determine which objects are examined for consistency when DG's output is requested (via **File | Output...**). Selecting all of these checks is recommended. Users should override them only if they are aware of the consequences of the object in question being improperly defined.

6.4 Creating and editing surfaces

As was noted in Sec. 1.2.2, surfaces are drawn between targets and are bounded by structures. The target related variables are single elements set using the variables **Variables | Inner edge of inner target**, etc., where the first “inner” and “outer” refers to the inner and outer edge of the computational mesh, respectively, and the second to the usual inner and outer divertor target. There is an additional set of targets available for double null geometries; see: **Variables | Add | Extra targets for DN**. The structures are set with the variable **Variables | Structure**. There may be one or more of these.

In Carre mode, the user begins the process of surface creation by manually creating the surface that will represent the innermost core surface via the *Add surface* tool. Just click it inside core plasma at the desired location (the tool will not work anywhere else). Its location can be altered via the *Move* tool. It

can only be deleted by executing the **Edit | Delete | Surfaces** command (this deletes *all* surfaces).

The next step is to open the **Create surface(s)...** dialog, **Edit | Create | Surfaces...** (see Fig 6.1). In Carre mode, the user can only create multiple surfaces (i.e., the “Single” option will not work). The “Area” menu determines in which part (topologically) of the equilibrium the surfaces will be drawn. The number of Cells must be input manually. The line graph in the dialog box is a simple depiction of the relative spacing of the surfaces. In Carre mode, only the “Delta Law” is available. In this case, the two parameters “Delta1” and “Delta2” specify a quadratic function used to determine this spacing (The deltas specify the relative normalized difference between the levels of the first two, and the last two surfaces).

While these parameters can be input directly, the preferred means of setting them is to click and drag the black line in the graph to achieve the desired spacing variation. When the “Create” button is clicked, the surfaces will be generated with this spacing and drawn in DG’s main window. If a model has an existing set of surfaces that the user wishes to tweak, click on one with the *Examine* tool and then click the “<<” button in the **Create surface(s)...** dialog; this will import the parameters describing these surfaces.

Note that the dialog will *not* enforce constraints on the number of cells (e.g., that the number of core and PFR cells must be equal); the user must check to be sure that all such constraints are satisfied.

DG’s algorithm for calculating the outermost flux surface level for unbounded regions (e.g., PFR and SOL) uses the **structure** and **inner / outer targets** variables. The only elements that the algorithm is aware of are those specified in the **structure** variable, shown in Fig. 6.1. Additionally, the algorithm recognizes at least two subsets of the structures, the “targets”, which consist of all closed loops in the structure on which elements in the **inner target** and **outer target** variables lie. All “target” elements must lie on a closed loop in order that the targets have a well defined “inside” and “outside”. The outermost surface level is then defined as the flux value differing most greatly from the value on the separatrix, whose corresponding flux surface passes through every target but encounters no other elements in the structure. This value is computed, to as much accuracy as the resolution of the equilibrium allows, as the least of 1) the global maximum on each target and 2) the global minimum on all other elements in the structure, where maximum and minimum are understood to be in terms of absolute deviation from the separatrix flux.

With this procedure, it is clear that the outermost level must be a flux ex-

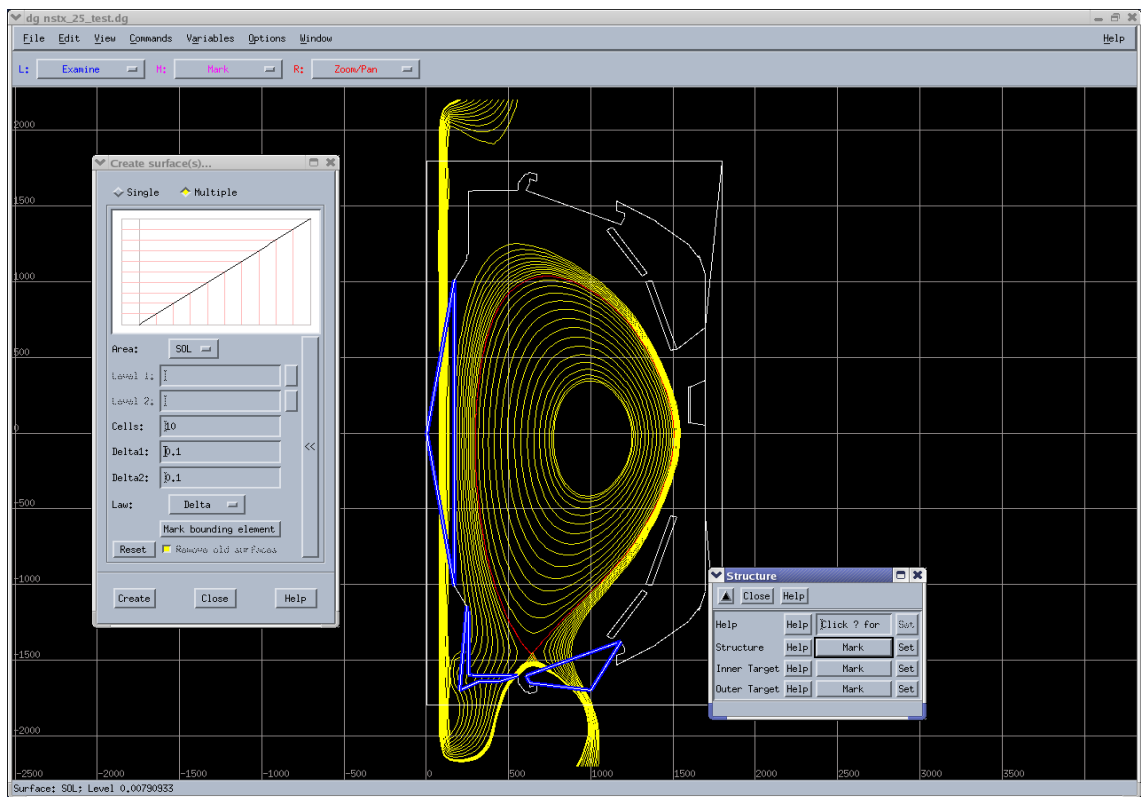


Figure 6.1: **Create Surface(s)...** dialog with highlighted Structures.

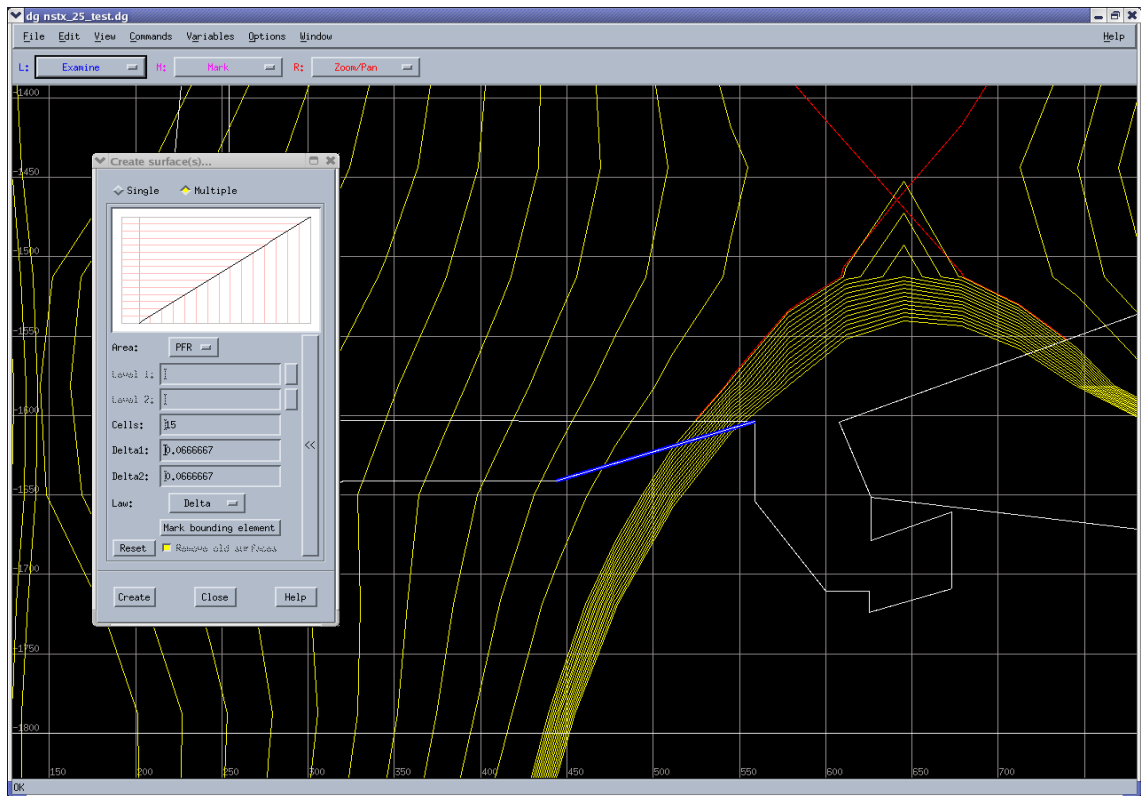


Figure 6.2: Zoom of the PFR showing the highlighted “bounding element”.

tremum of some element of the structure; we call this element the “bounding element”. The capability was added to DG to record this bounding element during surface creation and mark it from within the **Create Surface(s)...** dialog. See, e.g., Fig. 6.2. Note that the core region has no bounding element. By identifying what DG considers as the bounding element, we may easily adjust the extent of the flux surfaces by moving the bounding element to change the level of its extremum. Note that once the structure is changed, the bounding element might different than before.

6.5 Creating and editing grid points

The *Add grid point* tool lets you visually define grid points. In addition, the **Edit | Create | Grid point(s)** menu command lets you define a grid

point by specifying its value or fill a zone on the separatrix with a given number of grid points. In Carre mode, the only available method for creating grid points is with the “multiple” option of the **Create grid point(s)...** dialog box. In particular, the *Add grid point* tool will not work, nor will the *Move* or *Delete* tools have any effect on the grid points.

In Carre mode, the operation of this dialog box is directly analogous to that used to create surfaces (Sec. 6.4). The “zone” refers to the section of the separatrix along which grid points are to be created. The number of Cells must be input manually. The line graph in the dialog box is a simple depiction of the relative spacing of the grid points. In Carre mode, only the “Delta Law” is available. In this case, the two parameters “Delta1” and “Delta2” specify a quadratic function used to determine this spacing. While these parameters can be input directly, the preferred means of setting them is to click and drag the black line in the graph to achieve the desired spacing variation. When the “Create” button is clicked, the grid points will be generated with this spacing and drawn in DG’s main window. If a model has an existing set of grid points that the user wishes to tweak, click on one of these grid points with the *Examine* tool and then click the “i” button in the **Create grid point(s)...** dialog; this will import the parameters describing these grid points. Note that the dialog will *not* enforce constraints on the number of cells (e.g., that the number of grid points on either side of the PFR must be equal); the user must check to be sure that all such constraints are satisfied.

Use the **Edit | Delete | Grid points** menu command to delete all grid points.

Chapter 7

Working with chords

Chords are presently the only objects in DG that are not confined to the poloidal plane (i.e., that may be non-axisymmetric). For purposes of defining chords in three dimensions, two coordinate modes have been established. In “Cartesian” mode, Z is the vertical axis in the “top-down view” (Sec. 5.5); the horizontal axis in the poloidal plane is X and the vertical axis is Y (note: this is a left-handed coordinate system). In “Cylindrical” mode, Z is the vertical coordinate in the poloidal plane, identical to Y in “Cartesian” mode. In the “top-down view”, the horizontal axis is designated as R and the toroidal angle as ϕ (counterclockwise relative to the horizontal axis) is entered in degrees.

Chords can be displayed with either **View | Display | Chords** or **View | Display | 3dchords**. For chords confined to the poloidal plane, these are identical, and chords will not be visible in the “top-down view”. For chords with a toroidal extent, selecting **View | Display | Chords** in the poloidal plane results in straight line segments connecting the X and Y “Cartesian” coordinates of the chord end points. Equivalently in the “Cylindrical” mode, these are the R and Z coordinates at the end points. In contrast, **View | Display | 3dchords** shows the projection of the toroidal path of the chords on the $Z = 0$ (“Cartesian”) or $\phi = 0$ (“Cylindrical”) plane. In the “top-down view”, both **View | Display | Chords** and **View | Display | 3dchords** show the straight line paths of the chords in the X - Z (“Cartesian”) or $R \cos \phi$ - $R \sin \phi$ (“Cylindrical”) plane.

Starting with file format version 115, chords with X , Y , and Z components may be saved and loaded.

7.1 Creating chords

The *Add Chord* tool allows you to create chords graphically. To create a chord, move the mouse pointer to the desired location of its beginning and press the mouse button the tool is assigned to. Without releasing the mouse button, move the mouse pointer to the desired location of the other end of the chord and release the mouse button. “Chords” can also be created from marked “Elements” via the **Commands | Convert | Elements to chords** command. Duplicate chords are not produced and the original elements are deleted.

The third method for creating chords is with the **Edit | Create | Chord...** dialog.

7.2 Modifying chords

The *Move* tool allows chords to be manipulated in either the poloidal plane or in “top-down view”. The user can manipulate either the “Chords” object (displayed by **View | Display | Chords**) or the “3dchords” object (displayed by **View | Display | 3dchords**). The *Adjust chord* tool moves an end point of the chord while keeping its radial coordinate constant. Holding down the “shift” key while using the *Move* tool replicates this behavior, but with slightly different graphics that conform to the rest of the *Move* tool.

Chords have an orientation, just like elements; this orientation is displayed with **View | Display | External normals**. The *Reverse normals* tool can be used to change this orientation. Note that when this tool is dragged, it will only operate on objects of the same type as the first object encountered, either chords or elements.

The *Extend chord* tool linearly extends (i.e., replaces) the second endpoint of a chord to lie on the nearest element, or the nearest marked element if elements have been marked. The **Edit | Extend chords** command applies this for all marked chords. The **Edit | Mark all chords** command can be used to mark all chords in preparation for extension.

Note: as of August 2007, the “mathematically correct” procedure for extending chords has not been debugged. However, two macros defined in `chords.c` may be used to select a different preferred implementation of the procedure. `EXTCHORD_2D` selects the 2-D procedure, which does not support chords with nonzero toroidal extent, and `EXTCHORD_OLD` selects the partially correct, stable 3-D procedure, which does not work for horizontal chords and ignores horizontal elements. (This

is enabled in the current release of DG). If neither macro is defined, the code defaults to the incomplete procedure, which may show unpredictable behavior during use.

The **Edit** | **Delete** | **Chords** command deletes all chords.

Chapter 8

Working with sources

8.1 Creating and editing sources

DG allows the user to define sources and to attach variables to them.

To create sources, use the *Add source* tool. Move the mouse pointer to the desired location in the work area and press the mouse button. A source will be created at this location. You can move the mouse pointer to adjust its position. When finished, release the mouse button.

To create a source by entering its coordinates, select **Edit | Create | Source . . .**. A dialog box will appear prompting for coordinates. Enter values and press **Create**. A source will be created. The dialog box will remain on the screen, allowing you to define multiple sources without re-opening it. Press **Cancel** to close the dialog box.

To move a source to a new location, use the *Move* tool. Select the source to move, drag it to the new location and release the mouse button.

To delete a source, use the *Delete* tool. Select the source to delete and release the mouse button.

To delete all sources, issue the **Edit | Delete | Sources** menu command.

Chapter 9

Working with meshes

9.1 Importing a SONNET format mesh

DG allows you to import a mesh in the SONNET format. Carre’s output mesh can be converted into this format, as can meshes produced by other codes. The mesh will be displayed in the working area.

To import the mesh, use the **File | Import | Mesh...** menu command. A dialog box will appear prompting you for a filename. Select a file from the list or type the filename in the edit field. Then press **Load**. The mesh will be imported and displayed in the work area. If nothing is displayed, select **View | Picture view** to enlarge the area displayed. Also, make sure that **View | Display | Mesh** is enabled.

To hide or display the mesh, toggle **View | Display | Mesh**. To unload the mesh, select **Edit | Delete | Mesh**.

9.2 Modifying a mesh

DG’s graphical interface permits limited modification of a mesh that has been imported. The altered mesh can be written out to a text file (in the SONNET format) for use by other codes. The primary tool for this purpose is the “Move mesh point” tool. The actions of this tool are governed by the settings specified in the **Options | Mesh editing...** dialog. They only affect one-dimensional movements of mesh points.

“Slide along” specifies how movements of mesh points are confined. When “Magnetic surfaces” is in effect, the movement is confined to a magnetic surface

passing through the original location of the mesh point. This method should work fine except for the vicinity of the X-point, where magnetic surfaces drawn by DG become distorted. When “Splines” is in effect, the movement is confined to a spline passing through a row of existing mesh points. This method may be useful in the vicinity of the X-point.

“Margin” specifies the minimum distance between adjacent mesh points. If this limit is about to get exceeded, adjacent points start moving in order to “make room”. The “Double border” setting, when enabled, instructs DG to treat outermost mesh cells as a double border. They would be resized together with adjacent “regular” cells.

A modified mesh can be written out with the **File** | **Export** | **Mesh...** dialog.

9.3 Repairing mesh cells using rotated view and variable aspect ratio

Mesh cells highlighted in pink in DG’s main window are either concave (have an interior angle greater than 180 degrees) or consist of overlap with a neighboring cell. These problems typically arise where the mesh generator (Carre or otherwise) is forcing the mesh to conform to the specified divertor target shape and usually involve cells that are stretched significantly in one direction.

The ability to rotate and stretch the view allows the structure of the problem cells to be discerned and repaired. As an example, we consider a certain mesh generated for the NSTX experiment which contains twisted cells near the inner divertor, Fig. 9.1. By zooming, stretching, and rotating the view using the tools described in Secs. 5.3 and 5.4, we arrive at a much clearer depiction of the problem, Fig. 9.2. We see that the edges of the cells crossed the enforced boundary, twisting the mesh cells in the process. At this point, the user can straightforwardly use the *Move mesh point* tool to manually repair the cells.

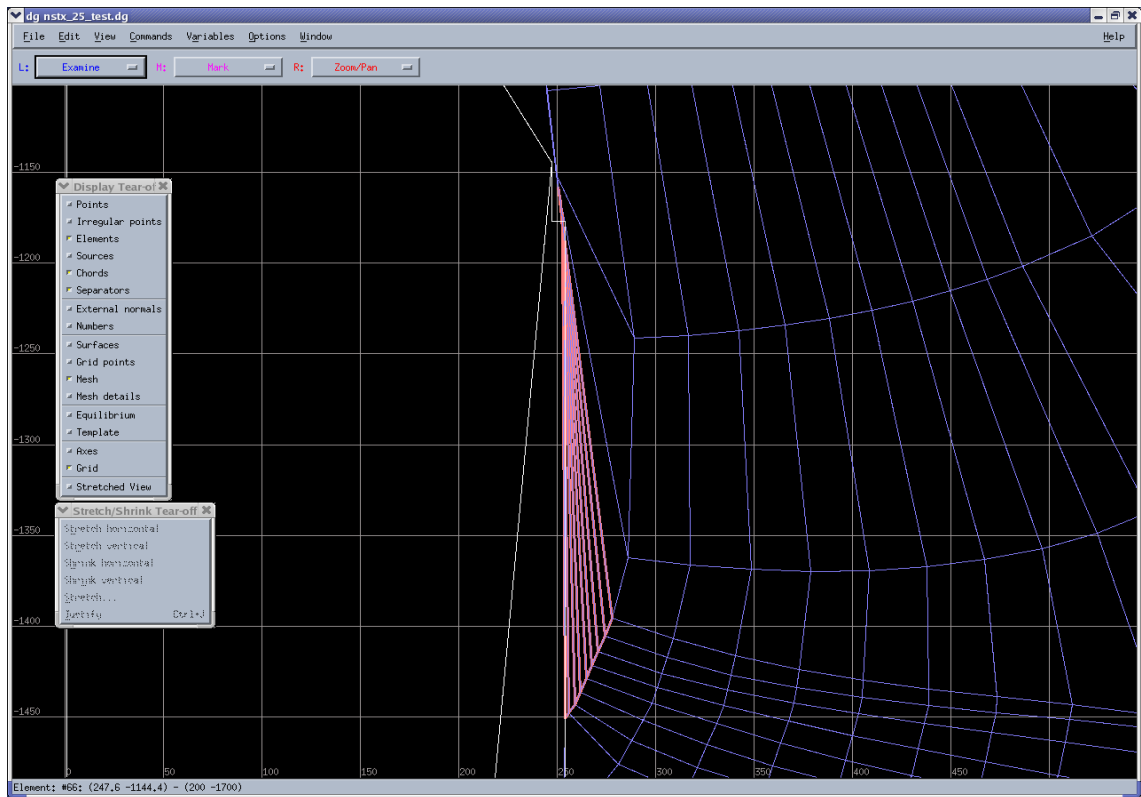


Figure 9.1: Zoom of the twisted mesh cells near the inner divertor target of an NSTX geometry.

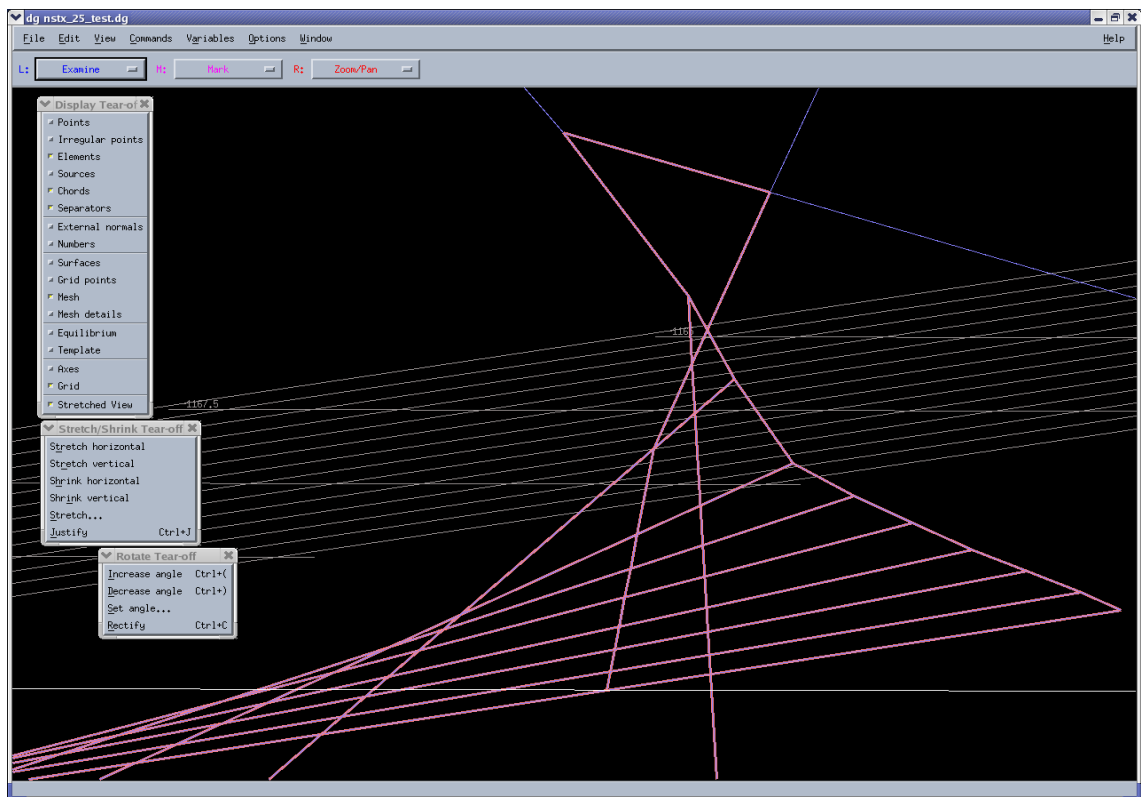


Figure 9.2: After stretching and rotation, the overlapping of the cells in the upper part of the region cells depicted in Fig. 9.1 becomes clear.

Chapter 10

Working with variables

10.1 Basics

With DG, you can use “Variables” to specify information that cannot be edited visually in the work area. Variables are written into the output file and can be processed by other programs. Some variables (e.g., those associated with **targets** and **structure**) also have important functions in DG. Variables are grouped together into “Layers”; each layer is visually represented in the code as a dialog box in which the constituent variables can be set, viewed, and modified.

Layers and their constituent variables must be defined by a power user before they can be instantiated and used in a particular DG model. This procedure is described in **Customizing DG**. Each variable has following attributes:

Name is written into the output file and is otherwise invisible.

Description is the name that is shown to the user.

Type defines type of values the variable can hold.

Scope specifies what objects the variable applies for.

Export/No export determines whether values are written into the output file.

Default value is the default value for newly-created objects.

Help text is displayed to the user when s/he asks for it.

These attributes are normally established by a power user while defining variables. A normal user should have some idea of the type and the scope in order to be able to set correct values for correct objects.

Each variable has a **type**. A variable can hold either a text value (**integer**, **real**, **text**, or **filename**) or a set of elements (**element**, **set of elements**, **target**, **structure**, **part of structure**). DG prevents the user from entering values of a wrong type.

The **scope** determines objects the variable is attached to. There are 4 possible scopes:

Layers means that the variable has one value for the whole layer.

Elements means the variable has a separate value for each element.

Sources means the variable has a separate value for each source.

Chords means the variable has a separate value for each chord.

As noted above, variables are organized into layers with DG displaying each layer in a separate dialog box.

10.2 Working with layers

Layers may have one or more instantiations in a particular model. For some layers, a single instance is all that is needed to specify the desired information; DG will not permit the creation of additional instances of this same layer. In the latter case, DG will assign an integer count as the instances are created. Some layer instances are predefined (e.g., **Structure**) and cannot be created or destroyed by the user, power or otherwise.

To create an instance of a specific layer, select the layer from the **Variables** | **Add** submenu. A dialog box for editing the variables in this layer instance will appear. If the maximum number of instances of a specific layer already exist, this layer will not be displayed in the **Add** menu. If this menu becomes empty, the **Variables** | **Add** cascade button will be grayed.

To destroy an instance of a layer, select its name in the **Variables** | **Remove** submenu. If there are multiple instances of a layer, there will be a submenu associated with that layer, listing the integer indices of those instances. If only the minimum number of instances of a specific layer already exist, layers of this type

will not be displayed in this menu. If this menu becomes empty, the **Variables** | **Remove** cascade button will be grayed.

To display the dialog box for an existing layer instance, select its name and instance index (if applicable) under the **Variables** menu.

To hide the dialog box for a layer, press the **Close** button in that dialog box.

10.3 Setting variables

The dialog box for each instance of a layer displays its variables, allowing them to be set, viewed, and modified. The title of this dialog box is the name of the layer; if there are multiple instances, the integer index of the particular index is also displayed here. At the top of the dialog box, there is a row of buttons that affect the entire box. Below this are fields for the individual variables. Finally, there is a status line at the bottom of the dialog box.

The “△” button on top of the dialog box allows you to “roll it up” so that the box occupies less screen space. In order to restore the dialog box to its full size, press this button again.

The dialog box can be resized using the window manager. If it is reduced in size, scrollbars appear, allowing you to scroll the content. The functionality of the dialog box is NOT affected by resizing it or rolling it up. In particular, the “Set all” button also affects variables that are invisible.

For each variable, the dialog box contains its name, a “Help” button, a text field for entering the new value (for text-like variables) or a “Mark” button (for “set of objects” variables), and a “Set” button. The “Help” button, when pressed, displays help about the variable. The text field is used to enter new values for text variables. When the text field has keyboard focus, the status line displays the current value of the variable. For text variables that apply to elements, chords or sources, only variables that apply to marked objects are accessed. “Filename” variables differ from “Text” in that the user can select the file interactively (press the “Enter” key on your keyboard in the text field to bring up the file selection dialog); the variable is not valid unless the file exists and is readable at the time of the variable check.

The “Mark” button, available only for “Set of objects” variables, marks the objects contained in the variable. The “Set” button sets the variable to the new value. For text variables, the new value must be entered in the text field. For variables storing sets of objects, the objects must be marked.

The “Set all” button in the top row, when pressed, has exactly the same effect

as pressing the “Set” button of each variable except that empty new values for variables that apply to elements, sources or chords are ignored. This button is only available if the layer consists of text variables only.

The “Reset all” button, when pressed, resets all text fields to present values of corresponding variables. It is only available in layers consisting of text variables only. The text field is cleared for variables that apply to elements, sources or chords when marked objects have different values.

The “Close” button closes the dialog box.

Pressing the right mouse button over a text field or a button associated with a variable brings up a pop-up menu. If the mouse has less than 3 buttons, the menu can be displayed by pressing “Ctrl+left” button. The “Reset” command sets the text field to the current value of the variable, similar to the “Reset all” button but affecting only one variable. For variables storing sets of objects, this command acts exactly like the “Mark” button. The “Display values” command displays values of variables that apply to elements, chords or sources next to corresponding objects. In order to remove these “labels”, highlight any object or use the **View | Remove labels** command from the main menu. Commands in the “Compare” submenu compare values of the variable to the value entered in the edit field and mark matching objects. Both the “Display values” command and the “Compare” submenu are only available for variables that apply to elements, sources or chords.

Chapter 11

Creating output files

11.1 Types of output files

DG produces 3 output files:

Structure file contains the structure.

Targets file contains targets, surfaces and grid points.

Output file contains elements, sources, separators and the values of the variables associated with all of the layer instances for the current model.

Current version of DG requires structure, targets, surfaces and grid points to be defined. Also, all **integer** and **real** variables must be set to non-empty values.

11.2 Producing output files

DG generates names of output files by attaching different extensions to the name of the DG file. If the current file has no name yet, save it with the **File | Save . . .** command before producing the output.

To produce the output files, choose the **File | Output** menu command. A dialog box will appear asking for confirmation. Press **Ok** to create the files.

Chapter 12

Customizing DG

12.1 Installing DG

To install DG, you need to:

- Compile and link its sources (additional guidance on how to do this with the version of DG distributed with DEGAS 2 are in Sec. 12.6).
- Make sure that the help file exists and has the right name
- Make sure that the configuration file exists and has the right name

DG is an X Windows application. The correct `DISPLAY` setting is necessary.

12.2 Files needed by DG

dg Executable file

dg.dgh Help file

dg.dgc Configuration file

12.3 Specifying directory masks

You can customize directory masks DG displays in file selection dialog boxes by setting following environment variables:

DGOPENMASK Directory mask for **File** | **Open...**

DGSAVEMASK Directory mask for **File** | **Save...**

DGEQUILMASK Directory mask for **File** | **Equilibrium...**

DGTEMPLATEMASK Directory mask for **File** | **Template...**

DGSONNETMASK Directory mask for **File** | **Sonnet grid...**

12.4 The configuration file

There is a special DG file, the **configuration file**. Its filename is normally produced by appending **.dgc** to the name of the executable. The configuration file has two functions:

1. When the **File** | **New** command is executed, the configuration file is loaded into the new window.
2. When a DG file is being opened, the configuration file is used to update its definitions of variables and layers.

DG normally writes these definitions in DG files. When a DG file is opened, they are normally ignored and definitions from the configuration file are used instead. This ensures that all files loaded in DG have identical definitions of layers and variables. Default values are not updated in the current version of DG.

12.4.1 Specifying the configuration file

Sometimes it is necessary to use a different configuration file or not to use any at all.

To use a different configuration file, start DG with the following command:

dg -cfg < filename-with-extension >

To use DG without loading a configuration file, type

dg -nocfg

NOTE: There are some bugs in DG that appear only when it is started without any configuration and sometimes cause an exception.

12.5 Creating or modifying the configuration file

Use the **Options | Setup** menu to create or modify the configuration file. Choose the **Variables...** submenu in order to edit definitions of variables and layers in the current file. The **Layer types** dialog box will appear.

Press **Save...** to save the current file as a configuration file. A dialog box will appear prompting for a filename. Normally, it already contains the right name. Select **Ok** to save the file in the current window as configuration. Press **Close** to close the dialog box.

12.5.1 Adding, modifying and removing layer types

To define a layer type, choose **Add** in the **Layer types** dialog box. The new **Layer type options** dialog box will appear.

Enter the name of the layer type to create in the **Name** text field. The name should contain only alphanumeric characters. This name will be written into the output file.

Enter the description in the **Description** text field. In the **MinInstances** / **MaxInstances text** field, enter the minimum and the maximum number of layers of this type. You will have to use 0 for **MinInstances** because there are no layers of this type yet. You can change **MinInstances** later after creating the minimum number of layers.

Press the **Ok** button to create the layer type.

To remove a layer type, select it in the list and press **Delete**.

To modify a layer type, select it in the list and press **Modify**. The **Layer type options** dialog box will appear. You can change name, description, Min/MaxInstances here. Note that if you change the name, layers contained in DG files under the old name will be ignored.

To close the **Layer types** dialog box, press **Close**.

12.5.2 Setting up variables

To define the variables that will constitute a layer, first display its **Layer type:** dialog box (“Modify”). This dialog box contains a rectangular area with buttons. These buttons represent row/column positions of variables in dialog boxes. Buttons containing the string **Empty** mean no variable is defined at this position.

To create a variable at a specific position, press the **Empty** button at that position. The **New variable** dialog box will appear.

Enter the name of the variable to be created in the **Name** field. The name must contain only alphanumeric character.

Select the type and the scope in corresponding option menus. Check the **Do not export** toggle button to prevent the variable from being written into the output file. Structure and targets are normally not exported because they are written into separate files.

Press **Ok** to create the variable.

To modify a variable definition, press its button in the **Layer type:** dialog box. The **Variable:** dialog box will appear. After making changes, press **Ok**. Note that if you change the name, variables contained in DG files under the old name will be ignored.

To remove a variable, press its button in the **Layer type:** dialog box. The **Variable:** dialog box will appear. Press **Remove** to remove the variable.

12.5.3 Adding help text to variables

DG lets you add help text to variables. The user can display the help text by pressing the **Help** button located near the name of the variable in the **Variables...** dialog box.

To edit a help text for a variable, display its **Variable:** dialog box and press **Edit help**. The **Help for variable** dialog box with a text editing field will appear. Edit the help text in the editing field and press **OK**.

12.5.4 Setting default values

DG lets you define default values for variables. When a new layer, element or source is created, the default values are used for the newly created object.

It is suggested that you define default values for all variables because DG will not generate output files when some variables have empty values. A search for empty values can be very difficult and cost a lot of time.

To define default values for variables in a layer, display its **Layer:** dialog box and choose the variable for which you wish to set a default. In its **Variables:** dialog box there is a “Default value” field for this purpose.

Default values from the configuration file are used only to create new DG files. When opening an existing file, default values from the configuration are ignored.

12.6 Compiling DG for use with DEGAS 2

1. In the `degas2/dg_carre/DG/class` directory, create a subdirectory representing the particular experiment you will be working with, e.g., `mkdir nstx`, creating the DG “nstx” run directory.
2. In `degas2/dg_carre/class/nstx` (or equivalent for your situation), do: `cp ../../dg .`, copying the DG shell script to the run directory.
3. In `degas2/dg_carre`, type: `source setup.csh nstx` (or equivalent for your situation). This sets up pointers to DG’s “nstx” run directory.
4. In `degas2/dg_carre/mscl`,
 - (a) Create a run directory for the specific type of computer being used (see the Makefile for supported names), e.g., `mkdir LINUX`,
 - (b) In that run directory, create a blank dependency file (to temporarily fool the Makefile), e.g., `touch dependencies.LINUX`,
 - (c) In the `mscl` directory, do: `gmake depend`.
 - (d) Then, `gmake`.
 - (e) These are miscellaneous routines used by DG and Carre.
5. In `degas2/dg_carre/DG`,
 - (a) Create a run directory, e.g., `mkdir LINUX`,
 - (b) In the DG directory, do: `gmake depend`.
 - (c) Then, `gmake`.
 - (d) In the run directory, create a link to the help file.
E.g., `ln -s ../src/dg.dgh dg.dgh`
 - (e) Also in the run directory, create a link to the configuration file:
`ln -s ../dg.dgc dg.dgc`
6. In `degas2/dg_carre/DG/equitrn`,
 - (a) Create a run directory, e.g., `mkdir LINUX`,
 - (b) In that run directory, create a blank dependency file (to temporarily fool the Makefile), e.g., `touch dependencies`,

- (c) In the `equtrn` directory, do: `gmake depend`.
- (d) This will create `.f` (and perhaps `.dbg`) files in the `equtrn` directory. These are not needed after this step and can be deleted.
- (e) Then, `gmake`.
- (f) These routines are used to translate equilibrium data files from one format to another (see Sec. 6.1).

7. In `degas2/dg_carre/Carre`,

- (a) Create a run directory, e.g., `mkdir LINUX`,
- (b) In that run directory, create a blank dependency file (to temporarily fool the Makefile), e.g., `touch dependencies.LINUX`,
- (c) In the `Carre` directory, do: `gmake depend`.
- (d) Then, `gmake`.

12.7 Example application of DG & Carre with DEGAS 2

This example workflow illustrates the application of the combined DG, Carre, and DEGAS 2's `definegeometry2d` codes for a single problem.

1. Prepare equilibrium data

- EFIT equilibrium data usually consist of 3 files, having names beginning with “a”, “g”, and “k”. This letter is followed by additional information that somehow encodes the shot number or date and time during the shot (this varies from one machine to the next). DG only needs the “g” file.
- Copy the desired “g” file to `degas2/dg_carre/DG/equtrn`. To avoid confusion, add `.efit` to this file name.
- Run `e2d` on this file, e.g.,
`e2d g112811.00490.efit g112811.00490.equ`
 The translated `.equ` file is in a format that DG can read.

- Copy the resulting file to the DG run directory, e.g.,
degas2/dg_carre/DG/class/nstx. Moving both the `.efit`
and `.equ` files will help prevent the `equtrn` directory from getting
cluttered. E.g.,
`mv g112811.00490.e* ../class/nstx/`

2. Run DG

- Change to the DG run directory. The `setup.csh` script creates an
alias for this with the name `cddg`.
- The DG documentation describes procedures for setting up the geom-
etry of an experiment from scratch. Instead, the following describes
modifications to an existing geometry, creating a new mesh from the
equilibrium and matching it up to the hardware (“elements” in DG).
E.g., begin by running on the command line:
`dg nstx_25.dg`
- Check the **Options | Output Mode...** dialog to be sure that
the **Carre** radio button is selected.
- Under DG’s **Edit** menu, **Delete** the Mesh and Equilibrium.
- Then, use **File | Import | Equilibrium** to load the new equi-
librium file.

3. Create plasma mesh

- DG is used to graphically set up the input for Carre. Carre is run
explicitly on the command line and the resulting mesh is loaded into
DG.
- The first step is to create the flux surfaces that Carre will use. Choose
View | Display | Surfaces to see if there are any surfaces al-
ready defined.
 - If none are seen, use the **File | Import | Topology** dialog
to import the topology corresponding to this equilibrium (e.g., SN
for lower single null).
 - If there are existing surfaces, use **Edit | Delete | Surfaces**
to delete them.
- In DG’s Carre mode, one uses the *Add surface* mouse button to
create a single surface in the core of an X-point equilibrium.

- The **Edit | Create Surface(s)...** dialog is then used to generate the rest.
 - A typical number of cells for the Core might be 26 with 19 in the SOL. The number in the PFR must match that of the Core.
 - Make a note of the number of cells used in the Core; this will be useful in setting up the input file for DEGAS 2's **defineback** input file.
 - The values of "Delta1" and "Delta2" can then be varied, either by hand or by tweaking the graphical display in the dialog box.
 - The particular objective depends on the problem at hand. E.g., it might be to get the spacing of surfaces near midplane roughly uniform going across the separatrix about 1 mm apart.
- Similarly, the **Edit | Create grid point(s)...** dialog is run to set the spacing of the mesh along the separatrix.
 - Be sure to select **View | Display | Grid points**.
 - The dialog box works just like the one for **Create surface(s)**.
 - Again, the number and spacing of the cells depends on the problem at hand.
- To prepare to run Carre:
 - Do **File | Save** to save the .dg file,
 - **File | Output** to create the files Carre will use,
 - Create the links to those files using the `lns` script. The argument for this script is the root of the DG file name. E.g., for `nstx_25.dg`, we would do on the command line:
`lns nstx_25`
- Run Carre
 - Change to the Carre directory with the alias `cdcarre`
 - On the command line, type:
`carre -`
 - If all goes well, one can use the default response (in parentheses) to each of Carre's queries and "y" to the yes / no questions.
 - Make a note of the number of the file containing the resulting grid. This will be stored in DG's run directory.
- Load mesh into DG with the **File | Import | Mesh** dialog.

- Note that this dialog box initially looks for files ending in `.sno`. Replace this `*.sno` with `***` and click the **Filter** button.

4. Create DG's polygons

- The first step here is to decide how much of the mesh will be used in the DEGAS 2 simulation. That is, `definegeometry2d` allows one to use only a contiguous subset of the entire mesh generated by Carre.
 - Note that the outermost mesh cells are very narrow and can only be discerned by zooming in. These are used to set boundary conditions in fluid plasma transport codes.
 - Note also that both DG and the Carre-generated mesh file (this is referred to as the “Sonnet format” file) begin numbering mesh cells at 0. In contrast, DEGAS 2's `definegeometry2d` starts its numbering at 1, so add 1 to the indices noted in DG.
 - As on the outer edge of the mesh, there is also a very narrow “ghost” cell at the innermost edge.
- The objective in defining polygons for use with `definegeometry2d` is to have them fill the entire problem space. To facilitate that, begin by defining a rectangular box (with **Edit | Create | Point...**) with corners exactly matching the box defined by the `bounds` keyword in `definegeometry2d`. That these match exactly is crucial.
- Existing polygons can be examined via the **Variables | Polygon | (polygon number)** dialogs.
 - The “Wall” component of the polygon refers to a set of contiguous elements that will form most or all of the polygon. The **Mark** button can be used to show the current definition. Its value can be changed by marking or unmarking elements and then clicking the **Set** button.
 - The **Material** component is currently not used and can be ignored.
- Polygons consisting only of a “Wall” (closed polygons) are complete.
 - The “First Mesh Connection” and “Second Mesh Connection” components need not be specified.
- However, some polygons will be closed by connecting the “Wall” elements to an edge of the mesh and following along that edge. The

points at which these connections are made and the edge to be included are specified by the “First Mesh Connection” and “Second Mesh Connection” components and the `iedge` keyword in `definegeometry2d`.

- Each “Mesh Connection” consists of the two mesh elements adjacent to the desired connection point, one going along the flux surface and one along the orthogonal. If the `sonnet_mesh` keyword in `definegeometry2d` will use a subset of the mesh, these elements need to be in that subset.
- Make a note of the corresponding mesh edge to use with the `iedge` keyword in `definegeometry2d`. These are denoted by S (for “south”, $i_z = 0$), E (“east”, $i_x = n_x$), N (“north”, $i_z = n_z$), or W (“west”, $i_x = 0$).

5. Finish with DG

- Again, save the file and do **File | Output**.
- Copy the resulting `.dgo` (not `.dg`) and Carre mesh files to the DEGAS 2 run directory.

Chapter 13

File formats

13.1 Simple template file format

Besides the "Neutral File" exported from PATRAN, DG can also import templates in a very simple format where the geometry is specified as a set of polygons (in general, open). In this case, each line of the input file either contains a pair of the co-ordinates of the next corner, or is empty.

13.2 Structure file format

The first line in the structure file contains the number of closed chains in the structure.

Then, for each chain, starting with the outermost, the following block appears:

<number of points in the chain>

<x1> , <y1>

...

For the outermost chain, the bounding rectangle is added.

Points are output in counterclockwise order when the external normals of the elements in the chain point outwards. Otherwise, they are output in clockwise order.

13.3 Targets file format

The targets file (for a single null model) has the following format:

```

target 1
line
  <x> , <y>
...
target 2
line
  <x> , <y>
...
region 1
levels
  <level>
...
region 2
levels
  <level>
...
region 3
levels
  <level>
...
zone 1
points
  <value>
...
zone 2
points
  <value>
...
zone 3
points
  <value>
...

<eof>

```

The **target1** and **target2** blocks contain point coordinates for targets.

The **region 1 . . . 3** blocks contain levels of surfaces. Regions have following meaning:

region 1 Main Plasma

region 2 SOL

region 3 Private Flux

The **zone 1...3** blocks contain values of grid points (between 0 and 1).
Zones have following meaning:

zone 1 Main chamber

zone 2 Inner divertor

zone 3 Outer divertor

13.4 Output file format

The output file has the following format:

```
userdata
p1
<x> , <y> , <z>
...
p2
<x> , <y> , <z>
misselem
<number>
...
sprtrs
<number>
...
chords
<xs> , <ys> , <xe> , <ye>
sources
<x> , <y>
<layer name> <layer instance number>
<variable>
...
...
finish
```

p1 and **p2** sections contain the coordinates of the origins and the ends of elements (respectively), ordered by element number. For unused element numbers, dummy coordinates are output. The Z coordinate is always 0.

The **misselem** section contains unused element numbers. Elements with these numbers **must** be ignored.

The **sprtrs** section lists numbers of separators in **p1** and **p2** arrays (currently disabled so that this section will be empty).

The **chords** section lists the X , Y coordinates of the start (“s”) and end (“e”) point of each chord. Note that the Z coordinate is *not* listed so that chords with a toroidal extent (see Sec. 7) are *not* properly described.

The **sources** section lists sources.

For each layer, a **<layer name>** **<layer instance number>** section is present. Layer name is the name of the layer. Layer instance number is used to distinguish between layers of the same type. Each section contains a **<variable>** entry for each variable in the layer.

The **<variable>** block has following formats:

- Text variables that apply to the layer: **<variable name>** **<value>**
- Text variables that apply to elements or sources:

```
<variable name>
<value>
...
```

- For variables that apply to elements, a value for each element number is included. Dummy values are used for unused numbers. For variables that apply to sources, a value for each source is included. Values are written in the same order as sources in the **sources** section.

- “Set of elements” variables that apply to the layer:

```
<variable name>
<element number>
...
```